

Model głębokiego uczenia do wieloklasowej detekcji ataków DDoS

1st Marcin Sadowski
Politechnika Warszawska
Warszawa, Polska
01178588@pw.edu.pl

2nd Maciej Kaniewski
Politechnika Warszawska
Warszawa, Polska
01178533@pw.edu.pl

Streszczenie—Wraz z rozwojem sieci definiowanych programowo (SDN) rośnie znaczenie skutecznych metod wykrywania ataków typu Distributed Denial of Service (DDoS). W literaturze coraz częściej wykorzystywane są modele głębokiego uczenia umożliwiające automatyczne wykrywanie złośliwego ruchu sieciowego. Jednym z takich rozwiązań jest model DDoSNet oparty o architekturę RNN-Autoencoder. Oryginalna wersja modelu realizuje jednak wyłącznie klasyfikację binarną, rozróżniając ruch normalny oraz złośliwy. Praca przedstawia rozszerzenie architektury DDoSNet do klasyfikacji wieloklasowej umożliwiającej rozróżnianie ataków DDoS. W ramach prac opracowano model oparty o warstwy liniowe oraz architekturę wykorzystującą sieci LSTM połączone z autoenkoderem. Modele zostały wytrenowane oraz ocenione z wykorzystaniem zbioru danych CICDDoS2019 [1] zawierającego współczesne typy ataków sieciowych.

Index Terms—SDN, DDoS, Deep Learning, LSTM, Autoencoder, Intrusion Detection System, CICDDoS2019, klasyfikacja wieloklasowa

I. WPROWADZENIE

Sieci definiowane programowo (SDN) zmieniają zarządzanie infrastrukturą poprzez separację warstwy sterowania od warstwy przesyłu danych. Umożliwia to centralizację zarządzania i większą elastyczność sieci, jednak wprowadza nowe ryzyka wynikające z centralizacji kontrolera. Jednym z najpoważniejszych zagrożeń są ataki DDoS, które poprzez przeciążenie zasobów destabilizują infrastrukturę i blokują dostęp do usług. W celu ich wykrywania coraz częściej stosuje się uczenie maszynowe i głębokie. Modele sieci neuronowych, takie jak DDoSNet (wykorzystujący architekturę RNN-Autoencoder), pozwalają na automatyczną klasyfikację ruchu i wykrywanie zależności między cechami ataków bez konieczności definiowania sztywnych reguł. Oryginalna implementacja modelu DDoSNet realizuje jednak wyłącznie klasyfikację binarną, rozróżniając ruch normalny od ruchu złośliwego. Takie podejście nie pozwala na identyfikację konkretnego rodzaju ataku, co ogranicza możliwości analizy zagrożeń oraz podejmowania odpowiednich działań obronnych. Celem niniejszej pracy jest rozszerzenie architektury DDoSNet do problemu klasyfikacji wieloklasowej umożliwiającej rozpoznawanie różnych typów ataków DDoS występujących w zbiorze CICDDoS2019 [1]. W ramach pracy opracowano model wykorzystujący głębokie sieci neuronowe oraz ulepszoną architekturę opartą o sieci LSTM i autoenkoder.

II. POWIĄZANE PRACE

Wzrost popularności sieci definiowanych programowo (SDN) znacząco zwiększył elastyczność zarządzania infrastrukturą, ale jednocześnie wystawił sieć na nowe typy zagrożeń, co zapoczątkowało badania nad zaawansowanymi metodami detekcji anomalii. Początkowo w systemach wykrywania intruzów dla środowisk SDN dominowały klasyczne metody uczenia maszynowego. Ye i in. [3] zaproponowali mechanizm detekcji ataków DDoS oparty na maszynach wektorów nośnych (SVM), wykazując wysoką skuteczność w analizie cech wyodrębnionych bezpośrednio z tabel przepływów. Podobnie Rahman i in. [4] zbadali zastosowanie tradycyjnych algorytmów uczenia maszynowego do wykrywania i mitygacji ataków w SDN.

Wraz ze wzrostem złożoności wzorców złośliwego ruchu, uwaga badaczy przeniosła się na modele głębokiego uczenia (Deep Learning), które charakteryzują się większą zdolnością do modelowania nieliniowych zależności. Tang i in. [5] jako jedni z pierwszych zastosowali głębokie rekurencyjne sieci neuronowe (RNN) do wykrywania intruzów w sieciach SDN, udowadniając, że architektury te są w stanie skutecznie przetwarzać sekwencyjny charakter ruchu sieciowego.

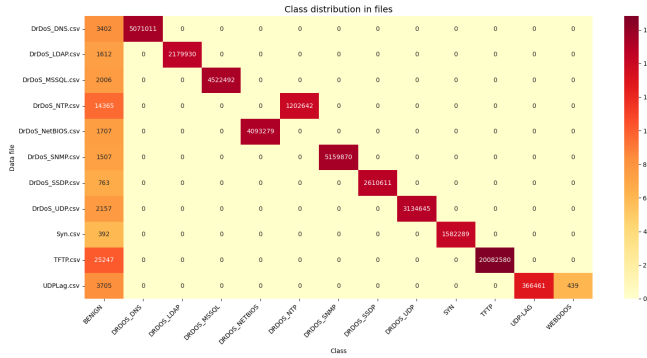
Bezpośrednią inspiracją dla niniejszej pracy jest model DDoSNet, opracowany przez Elsayeda i in. [2]. Rozwiązanie to z powodzeniem połączyło sieci RNN z autoenkoderem, osiągając wysoką precyzję detekcji zagrożeń.

III. OPIS DANYCH

W badaniach wykorzystano nowoczesny, publicznie dostępny zbiór danych CICDDoS2019 [1], opracowany przez Canadian Institute for Cybersecurity. Zbiór ten został wygenerowany w kontrolowanym środowisku emulującym rzeczywiste ataki sieciowe i stanowi jeden z najbardziej kompleksowych benchmarków w dziedzinie bezpieczeństwa. Dane zostały uprzednio przetworzone narzędziem CICFlowMeter, które z surowych plików PCAP wyekstrahowało ponad 80 wektorów cech statystycznych opisujących poszczególne przepływy, takich jak: liczba pakietów, rozmiary okien, czas trwania przepływu czy parametry przerw między pakietami.

Zbiór zawiera zarówno ruch poprawny (oznaczony jako *BENIGN*), jak i złośliwy, obejmujący różnorodne wektory ataków aplikacyjnych i wolumetrycznych (np. *SYN*, *UDP*, *TFTP*, *LDAP*, *MSSQL*, *NTP*, *SNMP*, *SSDP*, *UDP-LAG*, *WEBDDOS*).

Naturalną właściwością tego zbioru jest silne niezbalansowanie klas. Jak przedstawiono na rysunku 1, oryginalne pliki z poszczególnymi typami ataków zawierają od kilkuset tysięcy do nawet kilkudziesięciu milionów rekordów złośliwych (np. ponad 5 milionów w pliku *DrDoS_DNS.csv*), przy jednoczesnej śladowej ilości ruchu *BENIGN* występującej jako tło w każdym z plików. Tak ogromna dysproporcja wymagała podjęcia odpowiednich kroków przygotowawczych przed etapem trenowania modeli.



Rysunek 1. Oryginalny rozkład klas w poszczególnych plikach źródłowych zbioru CICDDoS2019 przed procesem balansowania (skala logarytmiczna dla kolorów).

IV. PRZETWARZANIE DANYCH

W celu zagwarantowania odpowiedniej jakości materiału uczącego, surowe dane poddano wieloetapowemu przetwarzaniu.

Pierwszym krokiem była **selekcja cech**. Z oryginalnego wektora danych celowo usunięto 10 atrybutów, które nie niosły wartości analitycznej lub mogłyby prowadzić do wycieku danych (ang. *data leakage*). Zbiór usuniętych cech obejmował:

- Atrybuty identyfikujące topologię sieciową i czas: *Flow ID*, *Source IP*, *Source Port*, *Destination IP*, *Destination Port* oraz *Timestamp*. Ich usunięcie jest kluczowe, ponieważ model miał uczyć się uniwersalnych wzorców i statystyk samego ataku, a nie zapamiętywać, z jakiego konkretnego adresu IP w laboratorium badawczym go przeprowadzono.
- *Unnamed: 0* – będący jedynie artefaktem po indeksowaniu wierszy z narzędzia ekstrakcyjnego.
- *Fwd Header Length.1* – będący zduplikowaną, nadmiarową kolumną wynikającą z błędu generatora CICFlowMeter.
- *SimilarHTTP* oraz *Inbound* – usunięte ze względu na zawartość danych tekstowych trudnych do reprezentacji numerycznej lub dostarczających trywialnych wskazówek (etykiety kierunku ruchu), które fałszywie zawyżałyby skuteczność detekcji modelu na tym konkretnym zbiorze.

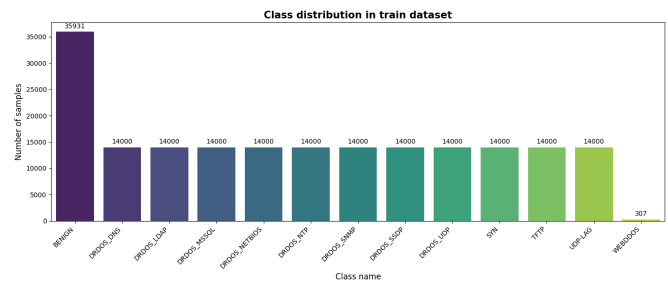
W efekcie tych operacji wektor wejściowy zredukowano do 77 atrybutów czysto numerycznych. Następnie przeprowadzono **oczyszczanie danych**, usuwając wszystkie przepływy posiadające brakujące wartości lub wartości nieskończone.

Kluczowym etapem było **balansowanie klas za pomocą podpróbkowania** (ang. *Undersampling*). Aby zapobiec zdominowaniu funkcji kosztu przez klasy milionowe (widoczne na rys. 1), nałożono globalny limit maksymalnie 20 000 próbek dla pojedynczej klasy ataku. Wyjątkiem była klasa *BENIGN*, która została zagregowana ze wszystkich plików źródłowych, oraz rzadkie klasy takie jak *WEBDDOS*, w przypadku których wykorzystano wszystkie dostępne próbki (niespełna 500 rekordów w całym zbiorze).

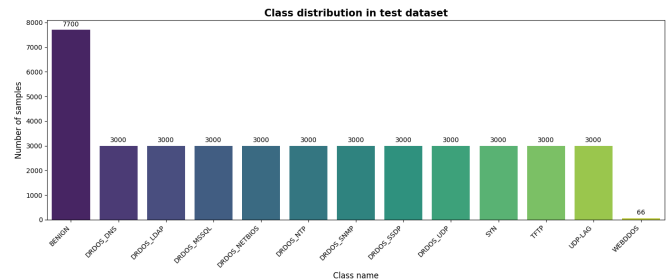
Przetworzony i przetasowany zbiór podzielono w proporcjach: 70% na zbiór treningowy, 15% walidacyjny oraz 15% testowy. Rozkład klas po podziale na zbiór treningowy oraz testowy zaprezentowano odpowiednio na rysunkach 2 i 3. Jak można zauważyć, zastosowane podpróbkowanie pozwoliło uzyskać równy udział większości klas ataków (równo po 14 000 próbek w zbiorze uczącym oraz 3 000 w zbiorze testowym).

Ostatnim krokiem była **normalizacja cech** (skalowanie Min-Max), z parametrami wyliczonymi na zbiorze treningowym i nałożonymi na pozostałe zbiory, oraz **kodowanie etykiet**, które zamieniło wartości tekstowe na całkowitoliczbowe identyfikatory od 0 do 12.

Na potrzeby eksperymentów z klasyfikacją binarną (reimplementacja modelu oryginalnego), wygenerowane etykiety poddano procesowi agregacji. Wszelkie klasy odpowiadające złośliwemu ruchowi zgrupowano w jedną wspólną kategorię (etykieta 1), podczas gdy ruch poprawny (*BENIGN*) zachował wartość 0. Takie podejście pozwoliło na spójne wykorzystanie tego samego, zbalansowanego zbioru danych w obu wariantach badawczych.



Rysunek 2. Rozkład klas w wygenerowanym zbiorze treningowym po zastosowaniu technik podpróbkowania (70% całości danych).



Rysunek 3. Rozkład klas w wygenerowanym zbiorze testowym (15% całości danych).

V. PROPONOWANA ARCHITEKTURA

A. Oryginalna architektura DDoSNet

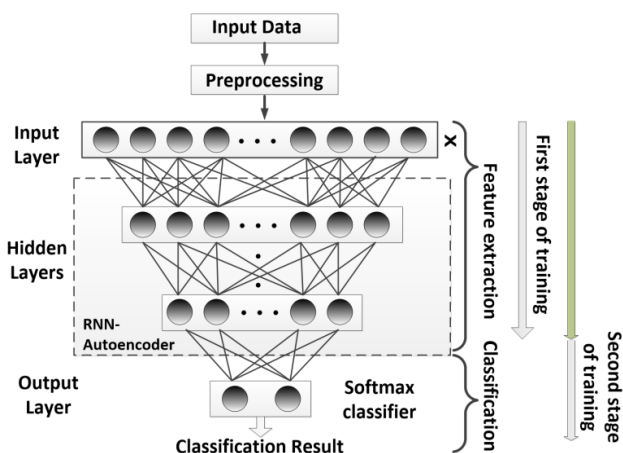
Podstawą niniejszej pracy jest model DDoSNet zaproponowany przez Elsayed i in. [2]. Model został opracowany do wykrywania ataków typu DDoS z wykorzystaniem technik głębokiego uczenia. Oryginalna architektura realizuje klasyfikację binarną, rozróżniając ruch poprawny (*BENIGN*) od ruchu złośliwego (*ATTACK*).

Architektura DDoSNet opiera się na połączeniu rekurencyjnych sieci neuronowych (RNN) z autoenkoderem. Autoenkoder odpowiedzialny jest za redukcję wymiarowości danych wejściowych oraz ekstrakcję najistotniejszych cech ruchu sieciowego.

Model składa się z dwóch głównych etapów:

- fazy uczenia nienadzorowanego,
- fazy dostrajania nadzorowanego.

W pierwszym etapie wykorzystywany jest autoenkoder oparty o warstwy RNN. Jego zadaniem jest nauczenie się skompresowanej reprezentacji danych wejściowych bez wykorzystania etykiet klas. Proces ten realizowany jest poprzez kodowanie danych wejściowych do przestrzeni o mniejszym wymiarze, a następnie ich rekonstrukcję w części dekodera. Część kodująca (*encoder*) składa się z czterech warstw ukrytych o malejącej liczbie neuronów: 64, 32, 16 oraz 8. Analogicznie część dekodująca (*decoder*) rekonstruuje dane wejściowe przy użyciu warstw o liczbie neuronów 8, 16, 32 oraz 64. Po zakończeniu procesu uczenia nienadzorowanego realizowany jest etap dostrajania modelu z wykorzystaniem danych oznaczonych. Na wyjściu modelu zastosowano warstwę klasyfikacyjną Softmax umożliwiającą klasyfikację danych do jednej z dwóch klas. Zaproponowana architektura została przedstawiona na rysunku 4.



Rysunek 4. Oryginalna architektura modelu DDoSNet oparta o RNN-Autoencoder [2].

B. Zaproponowana modyfikacja architektury

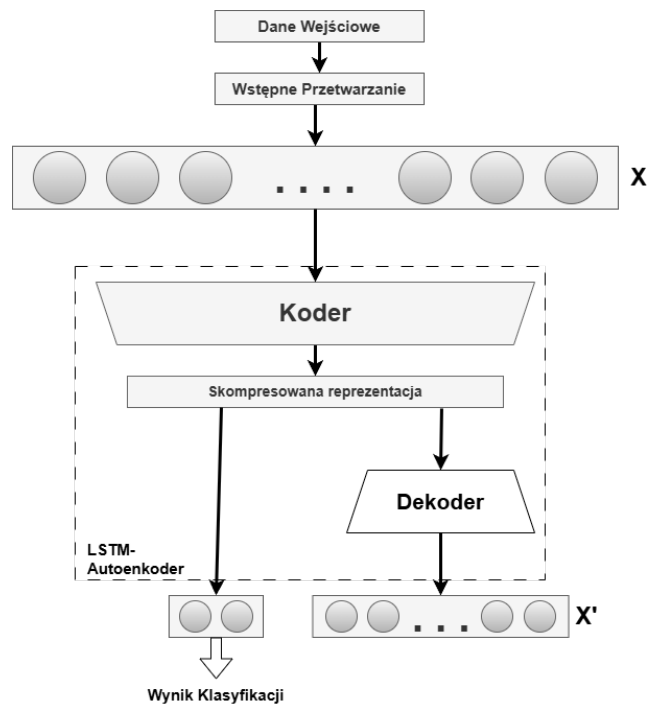
W ramach niniejszej pracy zmodyfikowano architekturę DDoSNet, adaptując ją do problemu klasyfikacji wieloklasowej. Podstawowa zmiana w module autoenkodera polegała

na zastąpieniu klasycznych bloków RNN warstwami LSTM (*Long Short-Term Memory*) [6], które skuteczniej radzą sobie z problemem zanikającego gradientu przy analizie sekwencji danych. Zachowano oryginalną strukturę wymiarowości: część kodująca (*encoder*) składa się z czterech warstw ukrytych (64, 32, 16, 8), a część dekodująca (*decoder*) odtwarza sygnał wejściowy w układzie zwierciadlanym.

W przeciwieństwie do oryginalnego podejścia, część klasyfikacyjna została dołączona bezpośrednio do wyjścia kodera (reprezentacja ukryta w przestrzeni 8-wymiarowej), a nie do dekodera. Proces uczenia podzielono na dwa odrębne etapy:

- 1) nienadzorowane trenowanie autoenkodera ukierunkowane na minimalizację błędu rekonstrukcji danych,
- 2) nadzorowane uczenie wieloklasowej warstwy klasyfikacyjnej na bazie cech wyekstrahowanych przez zamrożony (lub dostrajany) koder.

Zmiana ta pozwala na efektywne wykorzystanie silnie skompresowanych, najistotniejszych reprezentacji ruchu sieciowego do identyfikacji konkretnych typów ataków. Zaprojektowaną architekturę przedstawia rysunek 5.



Rysunek 5. zaprojektowana architektura oparta o LSTM-Autoencoder.

C. Architektura gęstej sieci neuronowej

Jako alternatywne podejście oraz punkt odniesienia dla modelu opartego na autoenkoderze, opracowano własną implementację głębokiej, gęstej sieci neuronowej (DNN, *ang. Deep Neural Network*). W tym przypadku zrezygnowano ze wstępnej redukcji wymiarowości za pomocą technik nienadzorowanych, model klasyfikuje ruch bezpośrednio na podstawie pełnego wektora cech wejściowych pochodzących ze zbioru CICDDoS2019.

Szczegółową strukturę warstwową zaprojektowanego modelu DNN przedstawiono w Tabeli I. Model składa się z trzech warstw liniowych rozdzielonych funkcjami aktywacji ReLU. W celu stabilizacji procesu uczenia oraz przyspieszenia zbieżności zastosowano warstwy normalizacji wsadowej (*BatchNorm1d*). Ponadto, aby zapobiec zjawisku przeuczenia (*overfitting*) i poprawić zdolności generalizacyjne sieci, wprowadzono warstwy (*Dropout*) z prawdopodobieństwem odrzucenia neuronów na poziomie odpowiednio $p = 0.3$ oraz $p = 0.2$. Ostatnia warstwa liniowa redukuje przestrzeń cech do 13 neuronów wyjściowych odpowiadających poszczególnym klasom ataków oraz ruchowi bezpiecznemu.

Tabela I
STRUKTURA WARSTWOWA ZAPROJEKTOWANEGO MODELU DNN.

Warstwa / Operacja
Wejście (Input: 77 cech)
Linear (77 → 256)
ReLU
BatchNorm1d (256)
Dropout ($p = 0.3$)
Linear (256 → 128)
ReLU
BatchNorm1d (128)
Dropout ($p = 0.3$)
Linear (128 → 64)
ReLU
Dropout ($p = 0.2$)
Wyjście (Linear: 64 → 13)

VI. KONFIGURACJA EKSPERYMENTÓW

W celu oceny efektywności zaproponowanych architektur przeprowadzono serię eksperymentów numerycznych. Podobnie jak w pracy oryginalnej [2], kluczowym elementem badania był wpływ współczynnika uczenia (*learning rate*, lr) na zbieżność procesów optymalizacji oraz końcową jakość klasyfikacji modeli.

Eksperymenty podzielono na dwa główne etapy:

- 1) **Reimplementacja modelu bazowego:** Ocena oryginalnej, binarnej architektury DDoSNet na współczesnym zbiorze danych CICDDoS2019 dla zestawu wartości $lr \in \{0.1, 0.01, 0.001, 0.0001\}$.
- 2) **Klasyfikacja wieloklasowa:** Ewaluacja zmodyfikowanego modelu LSTM-Autoencoder oraz referencyjnej gęstej sieci neuronowej (DNN) w zadaniu rozpoznawania konkretnych typów ataków oraz ruchu bezpiecznego.

Modele zostały zaimplementowane przy użyciu biblioteki PyTorch. Jako funkcję kosztu w klasyfikacji binarnej wykorzystano binarną entropię krzyżową (*Binary Cross-Entropy*), a w klasyfikacji wieloklasowej, wieloklasową entropię krzyżową (*Cross-Entropy Loss*). Do optymalizacji wag użyto algorytmu Adam. Jako metryki oceny modeli wyznaczono: dokładność (*Accuracy*), precyzję (*Precision*), czułość (*Recall*), miarę F1-score oraz pole powierzchni pod krzywą ROC (AUC) dla makro i mikro. Wszystkie testy końcowe przeprowadzono na tak samo podzielonym zbiorze danych.

VII. WYNIKI

A. Reimplementacja oryginalnego modelu

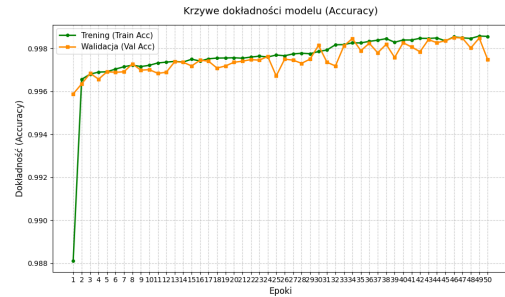
W pierwszej kolejności zbadano wpływ parametru lr na zbieżność binarnego modelu DDoSNet. W Tabeli II przedstawiono zestawienie najlepszych wyników uzyskanych na zbiorze walidacyjnym po odfiltrowaniu powtarzających się prób i niestabilnych konfiguracji dla skrajnych wartości parametrów.

Tabela II
WPŁYW WSPÓŁCZYNNIKA UCZENIA (lr) NA SKUTECZNOŚĆ BINARNEJ REIMPLEMENTACJI DDoSNET.

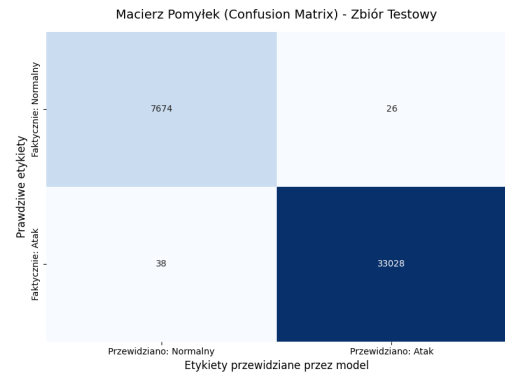
Współczynnik uczenia (lr)	Funkcja straty	Dokładność
0.001	0.009432	0.997571
0.0001	0.011473	0.997302
0.01	0.015648	0.996860
0.1	0.484658	0.811137

Najwyższa dokładność (0.9976) została osiągnięta dla parametru $lr = 0.001$.

Dla tej konfiguracji ponownie wytrenowano model dla większej ilości epok wynik **Accuracy = 0.9984**, gdzie wykres z uczenia został przedstawiony na rysunku 6. Model posiada niewielki margines błędu co można zobaczyć na rysunku 7 przedstawiającym macierz pomyłek. Szczegółowy raport klasyfikacji przedstawia Tabela III.



Rysunek 6. Wykres dokładności podczas uczenia modelu klasyfikacji binarnej



Rysunek 7. Macierz pomyłek dla klasyfikacji binarnej

Tabela III
RAPORT KLASYFIKACJI NA ZBIORZE TESTOWYM DLA BINARNEGO
MODELU DDoSNET ($lr = 0.001$).

Klasa	Precision	Recall	F1-score	Support
Normalny (0)	1.00	1.00	1.00	7700
Atak (1)	1.00	1.00	1.00	33066
Accuracy			1.00	40766
Macro avg	1.00	1.00	1.00	40766
Weighted avg	1.00	1.00	1.00	40766

B. Wyniki klasyfikacji wieloklasowej

W ramach eksperymentów nad klasyfikacją wieloklasową przeanalizowano zachowanie modelu opartego o autoenkoder LSTM oraz gęstą sieć DNN. Zestawienie wyników procesu strojenia hiperparametrów dla obu sieci zaprezentowano w Tabeli IV.

Tabela IV
PORÓWNANIE PARAMETRÓW UCZENIA DLA MODELI WIELOKLASOWYCH.

lr	Model LSTM-Autoencoder		Model DNN	
	Loss	Accuracy	Loss	Accuracy
0.1	2.443200	0.188863	13.645525	0.188618
0.01	0.580759	0.737348	0.681414	0.702833
0.001	0.513670	0.752533	0.604803	0.730602
0.0001	0.511064	0.755427	0.585294	0.745713

Dla modelu LSTM-Autoencoder oraz sieci DNN optymalną wartością okazał się parametr $lr = 0.0001$. Modele uzyskały dokładności odpowiednio na poziomie **0.7503** oraz **0.7444**, jak i wartości Micro AUC na poziomach **0.9870** i **0.9864**.

Szczegółowe zestawienie zdolności predykcyjnych w rozbiściu na poszczególne podklasy ataków przedstawiają Tabela V (dla modelu sekwencyjnego) oraz Tabela VI (dla sieci gęstej).

Tabela V
RAPORT KLASYFIKACJI DLA MODELU LSTM-AUTOENCODER.

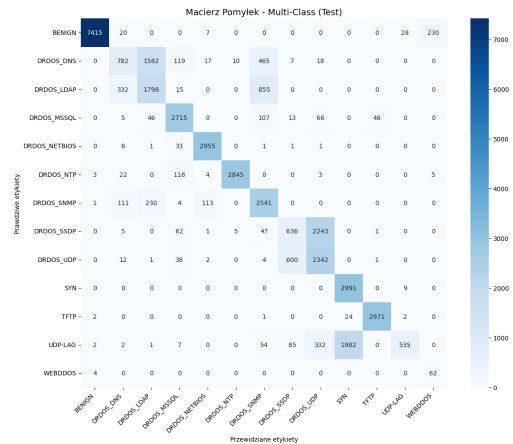
Typ ruchu / ataku	Precision	Recall	F1-score	Support
BENIGN	1.00	0.96	0.98	7700
DRDOS_DNS	0.60	0.26	0.36	3000
DRDOS_LDAP	0.49	0.60	0.54	3000
DRDOS_MSSQL	0.87	0.91	0.89	3000
DRDOS_NETBIOS	0.95	0.98	0.97	3000
DRDOS_NTP	0.99	0.95	0.97	3000
DRDOS_SNMP	0.62	0.85	0.72	3000
DRDOS_SSDP	0.47	0.21	0.29	3000
DRDOS_UDP	0.47	0.78	0.59	3000
SYN	0.60	1.00	0.75	3000
TFTP	0.98	0.99	0.99	3000
UDP-LAG	0.93	0.18	0.30	3000
WEBDDOS	0.21	0.94	0.34	66
Accuracy			0.75	40766
Macro avg	0.71	0.74	0.67	40766
Weighted avg	0.78	0.75	0.73	40766

Oba podejścia wieloklasowe wykazują bardzo wysokie wartości wskaźników obszaru pod krzywą ROC (odpowiednio

Tabela VI
RAPORT KLASYFIKACJI DLA MODELU GĘSTEJ SIECI DNN.

Typ ruchu / ataku	Precision	Recall	F1-score	Support
BENIGN	1.00	0.98	0.99	7700
DRDOS_DNS	0.48	0.77	0.59	3000
DRDOS_LDAP	0.52	0.03	0.05	3000
DRDOS_MSSQL	0.88	0.86	0.87	3000
DRDOS_NETBIOS	0.95	0.99	0.97	3000
DRDOS_NTP	0.99	0.95	0.97	3000
DRDOS_SNMP	0.62	0.84	0.72	3000
DRDOS_SSDP	0.47	0.03	0.05	3000
DRDOS_UDP	0.46	0.96	0.63	3000
SYN	0.60	1.00	0.75	3000
TFTP	0.94	0.99	0.97	3000
UDP-LAG	0.99	0.18	0.30	3000
WEBDDOS	0.32	0.92	0.48	66
Accuracy			0.74	40766
Macro avg	0.71	0.73	0.64	40766
Weighted avg	0.77	0.74	0.69	40766

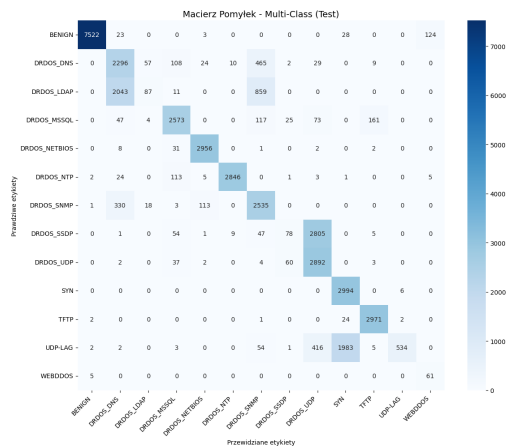
0.9757 oraz 0.9751 Macro AUC), co świadczy o wysokim potencjale dyskryminacyjnym sieci. Analiza szczegółowa ujawnia jednak trudność modeli w precyzyjnym różnicowaniu niektórych blisko spokrewnionych protokołów (np. ataki bazujące na SSDP oraz UDP-LAG charakteryzują się niskimi wartościami miary *Recall*) co można zaobserwować na macierzach pomyłek przedstawionych odpowiednio na rysunkach 8 oraz 9. Klasa *WEBDDOS*, pomimo bardzo małej liczebności w zbiorze testowym (*Support* = 66), została poprawnie zidentyfikowana w większości przypadków przez oba modele (*Recall* na poziomie 0.94 dla LSTM oraz 0.92 dla DNN).



Rysunek 8. Macierz pomyłek dla modelu LSTM-Autoencoder

VIII. WNIOSKI

W ramach niniejszej pracy przeprowadzono analizę możliwości wykorzystania metod głębokiego uczenia do wykrywania oraz klasyfikacji ataków DDoS w środowiskach SDN. Oryginalna architektura DDoSNet została pomyślnie odtworzona oraz rozszerzona z klasyfikacji binarnej do problemu klasyfikacji wieloklasowej obejmującej współczesne typy ataków



Rysunek 9. Macierz pomyłek dla sieci DNN

występujących w zbiorze CICDDoS2019. Uzyskane wyniki potwierdzają bardzo wysoką skuteczność modeli głębokiego uczenia w zadaniu detekcji złośliwego ruchu. Reimplementacja oryginalnego modelu binarnego osiągnęła dokładność przekraczającą 99%, co potwierdza przydatność architektury autoenkodera opartego o sieci rekurencyjne do identyfikacji anomalii sieciowych. W przypadku klasyfikacji wieloklasowej oba zaproponowane modele uzyskały zbliżone rezultaty. Dokładna analiza wyników pokazuje, że problem wieloklasowej identyfikacji ataków DDoS jest trudniejszym zadaniem. Modele osiągały wysoką skuteczność dla klas o wyraźnych charakterystykach ruchu, natomiast miały trudności z rozróżnianiem ataków opartych o podobne protokoły sieciowe.

LITERATURA

- [1] I. Sharafaldin, A. H. Lashkari, S. Hakak oraz A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," w *International Carnahan Conference on Security Technology*, 2019.
- [2] M. S. Elsayed, N.-A. Le-Khac, S. Dev oraz A. D. Jurcut, "DDoSNet: A Deep-Learning Model for Detecting Network Attacks," w *2020 IEEE 21st International Symposium on "A World of Wireless, Mobile and Multimedia Networks"(WoWMoM)*, 2020.
- [3] J. Ye, X. Cheng, J. Zhu, L. Feng oraz L. Song, "A DDoS Attack Detection Method Based on SVM in Software Defined Network," *Security and Communication Networks*, 2018.
- [4] O. Rahman, M. A. G. Quraishi oraz C.-H. Lung, "DDoS Attacks Detection and Mitigation in SDN Using Machine Learning," w *IEEE World Congress on Services*, 2019.
- [5] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi oraz M. Ghogho, "Deep Recurrent Neural Network for Intrusion Detection in SDN-based Networks," w *IEEE Conference on Network Softwarization*, 2018.
- [6] S. Hochreiter oraz J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.